2.4 FSM and FIFO Design Review

Overview: Following the initial drafting of the Day 2 FSM controllers and the synchronous FIFO, I conducted a self-review of the RTL code to identify potential synthesis issues, specifically looking for inferred latches, incomplete conditional branches, and unhandled states.

Issues Found and Fixes Applied:

- **Vending Machine Controller (Inferred Latches):**
    - *Issue:* In the initial draft of vending.v, the combinational always @(*) block did not assign a value to dispense, return_change, and refund in every possible execution path (e.g., when no coins were inserted). This would cause the synthesis tool to infer unwanted latches to hold the previous values.
    - *Fix:* I added default assignments (dispense = 0;, return_change = 0;, refund = 0;, and next_balance = balance;) at the very top of the combinational block. This guarantees that all outputs have a defined state every cycle, strictly preventing latches.
- **Sequence Detector (Missing Default Case):**
    - *Issue:* The Moore FSM in seq_moore.v uses a 3-bit state register, which has 8 possible states. However, the sequence detector only utilizes 5 valid states (0 through 4). Initially, states 5, 6, and 7 were left undefined in the case statement.
    - *Fix:* I added a default: next = 3'd0; branch to the state transition logic. This ensures that if the FSM ever enters an illegal state due to a glitch, it will safely reset back to the initial state rather than locking up.
- **Synchronous FIFO (Simultaneous Read/Write Collision):**
    - *Issue:* When reviewing the FIFO control logic, I realized that if a read and a write occurred in the exact same clock cycle (we = 1 and re = 1), the count register would be subjected to simultaneous increment and decrement commands, potentially corrupting the depth tracker.
    - *Fix:* I modified the control logic to use a concatenated case statement checking {we & ~full, re & ~empty}. I explicitly added a 2'b11 case where both the read and write pointers increment, but the count register is explicitly bypassed and remains unchanged.
- **Traffic Light Controller (Incomplete Reset):**

- *Issue:* During the review of traffic.v, I noticed the asynchronous pedestrian request flag (ped_pending) was not being cleared during the system reset.
- *Fix:* I updated the if (rst) block to explicitly force ped_pending <= 0; alongside the state and counter resets, ensuring a completely clean initialization.

- **ATM Controller (Incomplete if-else branches):**
  - *Issue:* In the ATM state machine, several conditional transitions (like checking if the PIN is correct) did not have an `else` branch specifying what happens to internal registers if the condition is false, which could lead to latches.
  - *Fix:* I added default assignments at the very top of the combinational `always @(*)` block (`next_state = state; next_balance = balance; next_attempts = pin_attempts;`). This ensures all registers maintain their current state safely unless explicitly overridden by the `if` conditions.